

ASSIGNMENT

Course: Object Oriented Programming through JavaScript

Assignment No.: 1

Student Name: Gorapalli Sravya

Roll No.: A24126552257

Date: 30-08-2026

1. TITLE

Implement Inheritance Types using JavaScript

2. AIM

To implement and demonstrate the different types of inheritance (Single, Multilevel, Hierarchical, Multiple, and Hybrid) supported in JavaScript using ES6 classes.

3. OBJECTIVE

The objectives of this assignment are:

- To understand the concept of inheritance in Object-Oriented Programming.
 - To implement Single Inheritance using the **extends** keyword.
 - To implement Multilevel Inheritance by chaining classes.
 - To implement Hierarchical Inheritance with multiple child classes.
 - To implement Multiple Inheritance in JavaScript using the Mixin pattern.
 - To implement Hybrid Inheritance combining more than one inheritance type.
 - To verify the working of each inheritance type using console output.
-

4. THEORY

Inheritance is an Object-Oriented Programming concept where a class (child/derived class) acquires the properties and methods of another class (parent/base class). JavaScript implements inheritance using prototypes, and ES6 introduced the **class** and **extends** keywords as syntactic sugar over prototypal inheritance.

JavaScript supports the following inheritance types:

- **Single Inheritance:** One child class inherits from exactly one parent class.
 - **Multilevel Inheritance:** A class inherits from a class that itself inherits from another class, forming a chain (grandparent → parent → child).
 - **Hierarchical Inheritance:** Multiple child classes inherit from the same parent class.
 - **Multiple Inheritance:** A class inherits behaviour from more than one source. JavaScript classes cannot extend more than one class directly, so this is achieved using the **Mixin** pattern (functions that take a base class and return a new extended class).
 - **Hybrid Inheritance:** A combination of two or more inheritance types used together in a single class hierarchy.
-

5. ALGORITHM / PROCEDURE

- 1 Create a base class **Animal** with a property **name** and a method **eat()**.
- 2 Create class **Dog** that extends **Animal** and adds **bark()** — demonstrates Single Inheritance.

- 3 Create class **Puppy** that extends **Dog** and adds **whine()** — demonstrates Multilevel Inheritance.
 - 4 Create class **Cat** that also extends **Animal** alongside **Dog** — demonstrates Hierarchical Inheritance.
 - 5 Create mixin functions **CanSwim** and **CanFly**; apply both to a **Duck** class extending **Bird** — demonstrates Multiple Inheritance.
 - 6 Create class **Pet** extending **Animal**, apply an **IsTrained** mixin, then extend it in **ServiceDog** — demonstrates Hybrid Inheritance.
 - 7 Instantiate objects of each derived class and call their inherited and own methods.
 - 8 Run the script using Node.js and verify each method call produces the expected output.
-

6. PROGRAM

inheritance.js

```
/* =====
Assignment 1: Implement Inheritance Types using JavaScript
Demonstrates: Single, Multilevel, Hierarchical,
Multiple (via Mixins), and Hybrid Inheritance
===== */

// ----- Base class -----
class Animal {
  constructor(name) {
    this.name = name;
  }
  eat() {
    console.log(`${this.name} eats food.`);
  }
}

/* =====
1. SINGLE INHERITANCE
One child class inherits from one parent class.
===== */
console.log("=== 1. SINGLE INHERITANCE ===");

class Dog extends Animal {
  bark() {
    console.log(`${this.name} barks.`);
  }
}

const dog = new Dog("Rex");
dog.eat();
dog.bark();
```

```

/* =====
2. MULTILEVEL INHERITANCE
A class inherits from a class that itself inherits
from another class (grandparent -> parent -> child).
===== */
console.log("\n=== 2. MULTILEVEL INHERITANCE ===");

class Puppy extends Dog {
  whine() {
    console.log(`${this.name} whines like a puppy.`);
  }
}

const puppy = new Puppy("Tommy");
puppy.eat();
puppy.bark();
puppy.whine();

/* =====
3. HIERARCHICAL INHERITANCE
Multiple child classes inherit from the same parent class.
===== */
console.log("\n=== 3. HIERARCHICAL INHERITANCE ===");

class Cat extends Animal {
  meow() {
    console.log(`${this.name} meows.`);
  }
}

const cat = new Cat("Whiskers");
cat.eat();

```

```

cat.meow();

const dog2 = new Dog("Buddy");
dog2.eat();
dog2.bark();

/* =====
4. MULTIPLE INHERITANCE (via Mixins)
JavaScript classes cannot extend more than one class
directly, so multiple inheritance is simulated using
mixin functions that return a class extending a base class.
===== */
console.log("\n=== 4. MULTIPLE INHERITANCE (using Mixins) ===");

const CanSwim = (Base) =>
class extends Base {
  swim() {
    console.log(`${this.name} swims.`);
  }
};

const CanFly = (Base) =>
class extends Base {
  fly() {
    console.log(`${this.name} flies.`);
  }
};

class Bird extends Animal {}
class Duck extends CanFly(CanSwim(Bird)) {}

const duck = new Duck("Donald");

```

```

duck.eat();
duck.swim();
duck.fly();

/* =====
5. HYBRID INHERITANCE
A combination of more than one type of inheritance
(here: single inheritance + mixin) in one hierarchy.
===== */
console.log("\n=== 5. HYBRID INHERITANCE ===");

class Pet extends Animal {
  play() {
    console.log(`${this.name} plays.`);
  }
}

const IsTrained = (Base) =>
class extends Base {
  obey() {
    console.log(`${this.name} obeys commands.`);
  }
};

class ServiceDog extends IsTrained(Pet) {
  guide() {
    console.log(`${this.name} guides its owner.`);
  }
}

const serviceDog = new ServiceDog("Max");
serviceDog.eat();

```

```

serviceDog.play();
serviceDog.obey();
serviceDog.guide();

```

7. CODE EXPLANATION

Code	Explanation
<code>class Animal { ... }</code>	Base class with a shared property (name) and method (eat).
<code>class Dog extends Animal</code>	Single Inheritance — Dog inherits from Animal.
<code>class Puppy extends Dog</code>	Multilevel Inheritance — Puppy inherits from Dog, which inherits from Animal.
<code>class Cat extends Animal</code>	Hierarchical Inheritance — Cat and Dog both inherit from the same parent, Animal.
<code>const CanSwim = (Base) => class extends Base</code>	A mixin function that adds <code>swim()</code> to whatever base class is passed in.
<code>class Duck extends CanFly(CanSwim(Bird))</code>	Multiple Inheritance — Duck combines behaviour from Bird, CanSwim and CanFly.
<code>class ServiceDog extends IsTrained(Pet)</code>	Hybrid Inheritance — combines single inheritance (Pet extends Animal) with the IsTrained mixin.
<code>super / extends</code>	Keywords used to link a child class to its parent class in ES6.

8. TEST CASES AND EXECUTION DATA

The script was actually executed with Node.js and the real console output was recorded (not assumed) for each inheritance type.

Test Case	Inheritance Type	Object Tested	Expected Result	Status
TC-01	Single	dog (Dog)	eat() and bark() both work	Pass
TC-02	Multilevel	puppy (Puppy)	eat(), bark(), whine() all work	Pass
TC-03	Hierarchical	cat (Cat), dog2 (Dog)	Both inherit eat() from Animal independently	Pass
TC-04	Multiple (Mixin)	duck (Duck)	eat(), swim(), fly() all work	Pass
TC-05	Hybrid	serviceDog (ServiceDog)	eat(), play(), obey(), guide() all work	Pass

9. OUTPUT

Output 1: Running the script (Node.js)

```
PS C:\FullStack\A24126552257> node inheritance.js
```

Output 2: Console output (actual, captured from execution)

```
=== 1. SINGLE INHERITANCE ===
Rex eats food.
Rex barks.

=== 2. MULTILEVEL INHERITANCE ===
Tommy eats food.
Tommy barks.
Tommy whines like a puppy.

=== 3. HIERARCHICAL INHERITANCE ===
Whiskers eats food.
Whiskers meows.
Buddy eats food.
Buddy barks.

=== 4. MULTIPLE INHERITANCE (using Mixins) ===
Donald eats food.
Donald swims.
Donald flies.

=== 5. HYBRID INHERITANCE ===
Max eats food.
Max plays.
Max obeys commands.
Max guides its owner.
```

10. RESULT

Thus, the different types of inheritance in JavaScript — Single, Multilevel, Hierarchical, Multiple (via Mixins), and Hybrid — were successfully implemented using ES6 classes and verified by executing the program with Node.js. All test cases produced the expected output.